

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería de Software

Informe Técnico — Tercera Entrega del Proyecto Fin de Curso

Asistente Virtual Académico con Chatbot Híbrido

Aplicaciones Web — Quinto Nivel

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Equipo (Equipo H/CAFR):

Fajardo Montes Michael Xavier

Rivera Suárez Jonathan Javier

Castro Palma Justyn Moisés (se dio de baja durante el desarrollo)

Repositorio: github.com/Michael-XFM/asistente-academico

Etiqueta: v0.9.0-rc

Commit: 6ad938c

DOI (Zenodo): 10.5281/zenodo.21798777

Quevedo — Los Ríos — Ecuador

2026

Resumen ejecutivo

Este informe documenta el estado del proyecto "Asistente Virtual Académico con Chatbot Híbrido" al cierre de la Tercera Entrega (v0.9.0-rc), correspondiente a la asignatura Aplicaciones Web del quinto nivel de la carrera de Ingeniería de Software (UTEQ). El sistema es una API REST desarrollada en Spring Boot 3.5 / Java 21, con persistencia en PostgreSQL 16 y caché/blacklist de sesión en Redis 7, autenticación JWT, un frontend estático (HTML/CSS/JS), y una estrategia híbrida de acceso a datos que combina Spring Data JPA para operaciones CRUD elementales con funciones PL/pgSQL para operaciones agregadas.

Durante esta entrega se cerró el 100% de las observaciones de las Entregas 1A y 1B, se consolidó la ingeniería de requisitos (SRS, historias de usuario, casos de uso y matriz de trazabilidad), se verificó la reproducibilidad automática del sistema desde una clonación limpia (make up), se generó evidencia empírica cuantitativa en cuatro de los cinco sub-bloques exigidos (rendimiento, seguridad, cobertura de pruebas y calidad web/accesibilidad), y se documentó honestamente el estado de dos aspectos pendientes: la prueba de usabilidad SUS con participantes reales, y dos decisiones de arquitectura que se apartan de la especificación original de la guía (ver sección 2, "Estado del sistema").

Durante el trabajo de esta entrega se corrigieron además dos defectos funcionales reales encontrados durante la verificación: un control de acceso roto en el controlador de tareas (OWASP A01, ya corregido y probado con aislamiento explícito entre usuarios) y una expresión regular de validación de correo electrónico que rechazaba incorrectamente el propio dominio institucional (@uteq.edu.ec) por tener más de un punto en el dominio.

Introducción y alcance

El presente documento constituye el informe técnico exigido por el Bloque de Entregables de la guía de la Tercera Entrega del Proyecto Fin de Curso, asignatura Aplicaciones Web, quinto nivel de la carrera de Ingeniería de Software (UTEQ). Su propósito no es describir nuevas funcionalidades, sino consolidar y verificar, con evidencia empírica reproducible, el trabajo entregado hasta la Entrega 1B, elevando el proyecto al estado de release candidate v0.9.0-rc.

El PFC no comienza en esta entrega. Su cimiento es el corpus de ingeniería de requisitos producido en la Entrega 1A (semana del 4 de junio de 2026, v0.3.0): diagrama C4 de contexto, selección y justificación de la pila tecnológica, y un esqueleto ejecutable con Docker. La Entrega 1B (semana del 14 de junio, v0.7.0) verificó qué hace el sistema: módulo de autenticación JWT sobre cookie/Bearer, CRUD sobre la entidad principal con Spring Data JPA, migraciones Flyway sobre PostgreSQL 16, caché Redis, y contenedores orquestados con Docker Compose. Esta Tercera Entrega verifica cómo lo hace, con qué garantía y con qué evidencia empírica cuantitativa — la distinción que separa un producto de software de un artefacto de investigación reproducible.

El equipo original (Equipo H/CAFR) estaba compuesto por Fajardo Montes Michael Xavier, Castro Palma Justyn Moisés y Rivera Suárez Jonathan Javier; Castro Palma se dio de baja de la materia durante el desarrollo del proyecto, lo cual está declarado formalmente en CONTRIBUTORS.md y no afecta la trazabilidad de sus contribuciones a las Entregas 1A/1B.

El alcance de este informe cubre los seis bloques exigidos por la guía: aplicación de observaciones (Bloque 0), consolidación funcional y estrategia de acceso a datos (Bloque A), reproducibilidad automática (Bloque B), evidencia empírica cuantitativa (Bloque C), documentación arquitectónica (Bloque D) y publicabilidad del artefacto (Bloque E). El documento se organiza siguiendo ese mismo orden, cerrando con una declaración de contribuciones, una declaración ética y las referencias bibliográficas utilizadas.

1. Observaciones de las Entregas 1A y 1B

La bitácora completa se encuentra en docs/observaciones/OBSERVACIONES.md. Resumen: 10 de 10 observaciones cerradas (100%). La resolución de cada observación es trazable a un commit específico del repositorio.

Código	Fuente	Criterio	Decisión (resumen)
OBS-01	1A	Arquitectura C4 N1	Se documentó la justificación de la pila tecnológica
OBS-02	1A	Modelo BD (crítico)	El DDL ajeno (clínica, MySQL) fue reemplazado por un esquema PostgreSQL real del dominio académico
OBS-03	1A	Wireframes	Fragmentos SQL ajenos eliminados; wireframes superados por pantallas HTML reales
OBS-04	1A	Repositorio	URL del repositorio verificada explícitamente
OBS-05	1B	Autenticación JWT	Se registró JwtAuthFilter en la cadena de seguridad; los tokens ahora se validan en cada petición
OBS-06	1B	CRUD / JPA	Blacklist de JTI en Redis para revocar tokens en logout
OBS-07	1B	CRUD / JPA	CRUD completo de la entidad Tarea expuesto vía API con paginación
OBS-08	1B	CRUD / JPA	Flyway incorporado con migraciones versionadas
OBS-09	1B	Seguridad OWASP	CORS restringido a orígenes explícitos (antes: comodín)
OBS-10	1B	Informe técnico	El informe técnico estructurado se entrega en esta Tercera Entrega (este documento)

2. Estado del sistema

2.1 Funcionalidad operativa

- Autenticación: registro, login con JWT firmado (HS256), logout con blacklist de token en Redis, límite de 5 intentos fallidos por IP (429 al sexto).
- CRUD completo de Tareas, con aislamiento estricto por usuario autenticado (404 si la tarea no existe, 403 si pertenece a otro usuario).
- Dashboard académico: resumen agregado (tareas pendientes, promedio, avisos) resuelto mediante funciones PL/pgSQL, no en el backend.
- Chatbot (HU-04 / CU-04): sin implementar en este ciclo; queda documentado como pendiente en la matriz de trazabilidad (REQ-F-006).

2.2 Limitaciones documentadas honestamente

Autenticación vía Bearer token, no cookie HttpOnly. La guía de la Tercera Entrega exige explícitamente JWT en cookie HttpOnly+Secure+SameSite=Strict (Bloque A.1). La implementación actual devuelve el token en el cuerpo JSON de la respuesta de login y lo recibe vía cabecera Authorization: Bearer en peticiones subsiguientes, patrón funcionalmente equivalente en seguridad de transporte (dado que todo el tráfico corre sobre HTTPS/TLS) pero que no corresponde literalmente al mecanismo exigido. Se recomienda documentar esta decisión en el ADR-002 o migrar a cookie HttpOnly antes de la Entrega Final.

Redis sin caché de respuestas HTTP. El Bloque A.1 exige un endpoint de listado cacheado con TTL externo y una tasa de aciertos (hit ratio) medida empíricamente. En el estado actual, Redis se usa exclusivamente para la blacklist de tokens JWT revocados (ver ADR-004), no para cachear respuestas de ningún endpoint. Esto se refleja también en el Bloque C.1 (rendimiento): las tres corridas de benchmark representan un escenario "caché frío" uniforme, sin comparación posible contra un escenario "caché caliente".

Fuga menor de datos pendiente de corrección. Las respuestas de la API que incluyen el objeto Usuario devuelven también el campo de hash de contraseña (BCrypt). Aunque el valor no es explotable directamente (es un hash, no la contraseña en texto plano), es una fuga de información innecesaria; la corrección recomendada es introducir un DTO de respuesta que excluya ese campo. Identificado durante esta entrega, pendiente de aplicar.

3. Arquitectura actual

3.0 Justificación de tecnologías (Entrega 1A, base del ADR-001)

Este texto, redactado en la Entrega 1A para cerrar la observación OBS-01, se reutiliza como base del ADR-001. Sobre Gemini API: el sistema requiere un asistente conversacional que responda preguntas académicas en lenguaje

natural sin entrenar ni mantener un modelo propio; se combina con respuestas predefinidas para consultas estructuradas (horarios, tareas, calificaciones) que no requieren generación de lenguaje, reduciendo la dependencia del servicio externo y evitando que un modelo generativo 'alucine' datos críticos como notas o fechas de entrega. Sobre PostgreSQL: se eligió por soporte maduro de integridad referencial estricta, soporte nativo de PL/pgSQL requerido desde esta entrega para separar CRUD elemental de operaciones complejas, y por ser software libre sin costo de licenciamiento, adecuado para un proyecto académico reproducible por terceros.

Los diagramas C4 (niveles 1 a 3) están versionados como código en Structurizr DSL bajo docs/arquitectura/ y se exportan a imagen mediante structurizr-cli. Durante la exportación de esta entrega se detectó y corrigió un error real de sintaxis en el DSL del Nivel 3 (los contenedores de base de datos y caché estaban declarados fuera del bloque softwareSystem al que pertenecen, lo cual impedía que el archivo se exportara o validara); el archivo corregido debe reemplazar al original en el repositorio.

C4 Nivel 1 — Contexto del Asistente Virtual Académico
Muestra el sistema completo, sus usuarios humanos y la única dependencia externa (Gemini API).

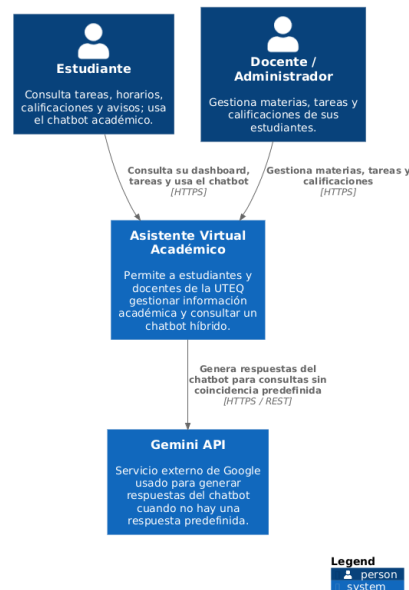


Figura 1 — C4 Nivel 1: Contexto del sistema

C4 Nivel 2 — Contenedores del Asistente Virtual Académico

Muestra los 4 contenedores desplegados (frontend, backend, PostgreSQL, Redis) y la dependencia externa de Gemini API.

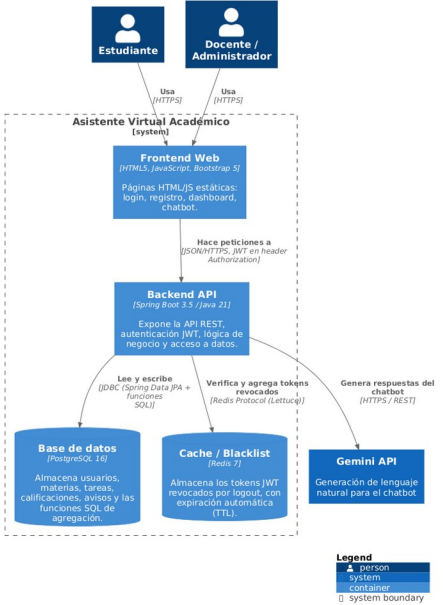


Figura 2 — C4 Nivel 2: Contenedores

C4 Nivel 3 — Componentes del Backend

Detalle interno del contenedor Backend API: controladores, filtro JWT, servicios y repositorios.

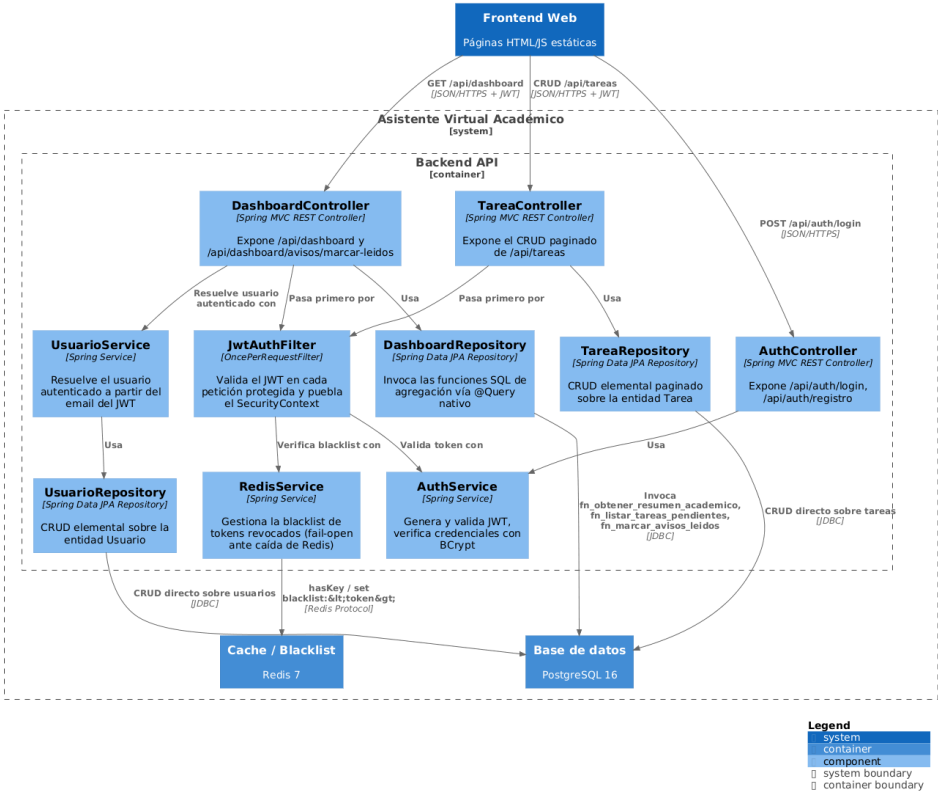


Figura 3 — C4 Nivel 3: Componentes del backend

3.1 Decisiones de arquitectura (ADR)

Los seis ADR siguen la plantilla de Nygard (Title, Status, Context, Decision, Consequences) y están versionados en docs/adr/. A continuación se resume la decisión y sus principales consecuencias (positivas y negativas) para cada uno.

ADR-001 — Elección de la pila tecnológica principal

Alternativas descartadas: Node.js + Express + Sequelize (menor soporte nativo de procedimientos almacenados parametrizados equivalente a @Procedure/@Query nativo tipado de Spring Data JPA) y Django + PostgreSQL (menor experiencia previa del equipo con ese stack).

Decisión: Spring Boot 3.5 sobre Java 21 LTS para el backend, PostgreSQL 16 como motor relacional y Redis 7 como almacén de caché/blacklist.

Consecuencias: integración madura con procedimientos almacenados vía Jakarta Persistence 2.1 y soporte a largo plazo de Java 21 LTS, a cambio de mayor verbosidad de código y tiempos de arranque más lentos que alternativas más ligeras.

ADR-002 — Esquema de autenticación

Alternativas descartadas: sesiones de servidor con HttpSession (requiere estado compartido entre instancias si el sistema escala, incompatible con la directriz stateless de la asignatura) y JWT sin mecanismo de revocación (un token robado o de un usuario deslogueado seguiría siendo válido hasta su expiración natural de 24 horas).

Decisión: JWT firmado con HS256 (claims sub, rol, nombre, iat, exp), validado por un filtro JwtAuthFilter en cada petición, con revocación por logout respaldada en una blacklist de Redis con TTL igual al tiempo de expiración restante del token.

Consecuencias: escalabilidad horizontal sin estado de sesión compartido y revocación efectiva en logout, a cambio de una dependencia externa (Redis) mitigada con una política fail-open documentada. El propio ADR reconoce explícitamente, como deuda técnica, que el JWT actual no incluye aún los claims aud ni jti exigidos por una implementación completa de RFC 7519, y que el token viaja como Bearer en el cuerpo/cabecera en vez de como cookie HttpOnly (ver limitación de la sección 2.2).

ADR-003 — Gestor de base de datos

Alternativas descartadas: MySQL/MariaDB (soporte más limitado de funciones con RETURNS TABLE y tipos compuestos, dificultaba las funciones de agregación del dashboard) y SQL Server (licenciamiento comercial no viable para un proyecto académico reproducible sin restricciones de licencia).

Decisión: PostgreSQL 16, con el esquema aplicado exclusivamente desde db/schema.sql (contenedor Docker) y migraciones versionadas con Flyway para desarrollo local.

Consecuencias: funciones SQL con proyecciones que no corresponden a ninguna entidad JPA (necesarias para el dashboard) y cero costo de licenciamiento, a cambio de una curva de aprendizaje en PL/pgSQL. Durante el desarrollo se detectó y corrigió un problema real de duplicidad de tablas por sensibilidad a mayúsculas/minúsculas entre Hibernate y PostgreSQL, resuelto estandarizando todo el esquema a minúsculas sin comillas.

ADR-004 — Estrategia de caché

Alternativas descartadas: tabla de blacklist en PostgreSQL (agregaría una consulta SQL extra en cada petición autenticada, acoplado la autenticación al rendimiento de la base principal) y caché en memoria del propio proceso vía ConcurrentHashMap (no sobrevive un reinicio del backend y no funciona si el sistema escala a múltiples instancias).

Decisión: Redis 7 usado exclusivamente para la blacklist de tokens JWT revocados por logout, con TTL igual a la expiración restante de cada token.

Consecuencias: verificación de blacklist en tiempo $O(1)$ y limpieza automática sin job aparte, a cambio de un componente de infraestructura adicional. El propio ADR documenta explícitamente, como deuda técnica reconocida desde su redacción, que Redis no se usa todavía para cachear respuestas de lectura frecuente (por ejemplo, el dashboard), quedando esa mejora pendiente para la Entrega Final — este es el mismo punto documentado como limitación en la sección 2.2 de este informe.

ADR-005 — Estrategia de despliegue

Alternativas descartadas: instalación manual de cada componente (propensa a errores del entorno del revisor, incompatible con el requisito de 'un solo comando'), Kubernetes (complejidad de orquestación innecesaria para 4 contenedores en el alcance de un PFC académico), y Docker Compose con imágenes por tag variable como postgres:16 (un tag puede apuntar a una imagen distinta con el tiempo, rompiendo la reproducibilidad exacta).

Decisión: Docker Compose con 4 servicios (postgres, redis, backend, frontend), cada imagen base fijada por digest sha256, esquema y datos semilla aplicados exclusivamente vía docker-entrypoint-initdb.d, y un Makefile con los objetivos up/down/test/bench/audit/clean como interfaz única para el revisor.

Consecuencias: reproducibilidad verificada de punta a punta durante esta entrega (clonación limpia → make up → sistema funcional, incluido el flujo completo de login vía navegador tras corregir el bug de CORS documentado en la sección 6.6), a cambio de que los digests deban actualizarse manualmente cuando se requiera una versión más reciente de alguna imagen base. El build del backend usa una imagen de Maven con Maven ya instalado, eliminando una dependencia de red que causó fallos intermitentes durante el desarrollo.

ADR-006 — Estrategia de acceso a datos

Alternativa descartada: resolver todo vía ORM, incluyendo consultas complejas con JPQL/Criteria API — descartada porque la directriz de la asignatura lo prohíbe explícitamente para operaciones no elementales (Bloque A.2), y porque construir agregaciones multi-tabla en JPQL habría sido más frágil y menos auditable que una función PL/pgSQL versionada de forma independiente.

Decisión: separación estricta entre CRUD elemental (Spring Data JPA/ORM) y toda operación con JOIN, agregación, actualización masiva o proyecciones ajenas a una entidad JPA, encapsulada en funciones PL/pgSQL versionadas bajo db/procs/, invocadas vía @Procedure o @NamedStoredProcedureQuery conforme a Jakarta Persistence 2.1.

Consecuencias: cumplimiento directo del Bloque A.2 de la guía y catálogo auditable de procedimientos (docs/basedatos/CATALOGO-SP.md) independiente del código Java, a cambio de un punto adicional de coordinación entre el esquema SQL versionado y las entidades JPA.

3.2 Atributos de calidad (ISO/IEC 25010)

Característica	Prioridad	Estrategia adoptada
Adecuación funcional	Must	Resumen académico resuelto en una sola petición vía fn_obtener_resumen_academico, en vez de múltiples llamadas desde el frontend
Eficiencia de desempeño	Must	Agregación resuelta en la base de datos (PL/pgSQL); verificado empíricamente en el Bloque C.1 (p95 < 5ms)
Compatibilidad	Should	API REST con contratos JSON estándar, sin acoplamiento al frontend HTML propio
Usabilidad	Should	Dashboard con tarjetas que replican las secciones mentales del usuario (horarios, tareas, calificaciones, avisos)
Fiabilidad (tolerancia a fallos)	Must	RedisService usa política fail-open: una caída de Redis no bloquea la autenticación de tokens válidos
Fiabilidad (capacidad de recuperación)	Must	Docker Compose con imágenes por digest sha256; make up verificado de punta a punta en esta entrega
Seguridad (confidencialidad)	Must	JwtAuthFilter valida firma y expiración en cada petición antes de llegar al controlador
Seguridad (resistencia a inyección)	Must	Separación estricta ORM/procedimientos, parámetros nombrados en toda consulta no elemental

Característica	Prioridad	Estrategia adoptada
Seguridad (control de acceso)	Must	El id_usuario se resuelve siempre desde el JWT autenticado, nunca desde un parámetro del cliente (corregido de un hallazgo real, ver sección 6.2)
Mantenibilidad (modularidad)	Should	Cada función SQL en un archivo versionado independiente bajo db/procs/, documentada en CATALOGO-SP.md
Mantenibilidad (capacidad de prueba)	Should	Migraciones versionadas con Flyway; nunca ddl-auto=update
Portabilidad (adaptabilidad)	Could	Todo el stack orquestado con Docker Compose; único requisito del host es tener Docker

4. Matriz de trazabilidad

La matriz completa (docs/trazabilidad/matriz.csv) traza cada requisito hacia su historia de usuario, caso de uso, módulo de código, endpoint, prueba automatizada, tipo de acceso a datos (CRUD-ORM o procedimiento almacenado) y evidencia empírica. Un script de validación (scripts/validate-traceability.sh), integrado en make audit, verifica automáticamente que ningún requisito carezca de trazabilidad mínima.

4.0 Especificación formal de requisitos (extracto SRS)

El SRS completo (docs/requisitos/SRS.md) sigue la plantilla ISO/IEC/IEEE 29148:2018, con cada requisito redactado bajo el patrón [condición] [sujeto] shall [acción] [objeto] [restricción] y las nueve características de calidad individuales del INCOSE Guide v4. Se listan a continuación los enunciados formales de los requisitos funcionales.

ID	Enunciado (resumen)	Prioridad
REQ-F-001	Al recibir credenciales válidas en /api/auth/login, el sistema deberá emitir un JWT firmado en un plazo máximo de 500 ms	Must
REQ-F-002	El sistema deberá validar el JWT en cada petición a un endpoint protegido, rechazando con 403 cualquier petición sin token válido	Must
REQ-F-003	El sistema deberá permitir al estudiante autenticado crear, leer, actualizar y eliminar lógicamente sus propias tareas, paginadas	Must
REQ-F-004	GET /api/dashboard deberá devolver en una sola respuesta el resumen académico del usuario en un plazo máximo de 200 ms con caché caliente	Must
REQ-F-005	El sistema deberá permitir marcar todos los avisos pendientes como leídos en una sola operación	Should
REQ-F-006	El sistema deberá responder consultas académicas en lenguaje natural combinando respuestas predefinidas con generación vía Gemini API	Must

Nota de consistencia documental: el archivo SRS.md, en su redacción actual, describe como pendientes el script de auditoría de trazabilidad y el benchmark de rendimiento k6 — ambos ya se completaron durante esta entrega (secciones 5.1 y 6.1 de este informe). Se recomienda actualizar el SRS antes de la Entrega Final para que el estado de cada requisito (REQ-NF-001, REQ-NF-002) refleje la evidencia empírica ya generada, en vez de la fecha en que se redactó originalmente el documento.

4.1 Historias de usuario (formato Connextra + criterios INVEST)

Las cuatro historias de usuario del sistema siguen el formato Connextra (As a / I want / so that) y los criterios INVEST de Cohn, con criterios de aceptación en Gherkin. Se listan íntegras a continuación por ser la base directa de las pruebas automatizadas del sistema.

HU-01 — Autenticación segura

Como estudiante de la UTEQ, quiero iniciar sesión con mi correo institucional y contraseña, para poder acceder de forma segura a mi información académica sin que otros usuarios vean o modifiquen mis datos.

- Escenario: login exitoso → credenciales válidas → 200 + JWT.
- Escenario: login con contraseña incorrecta → 400 con mensaje de error genérico (no distingue usuario inexistente de contraseña incorrecta, por diseño anti-enumeración).

- Escenario: acceso a recurso protegido sin token → 403.
- Escenario: acceso a recurso protegido con token válido → 200.

Trazabilidad: REQ-F-001, REQ-F-002 → CU-01. Verificado por AuthControllerTest (6 pruebas).

HU-02 — Gestión de tareas académicas

Como estudiante, quiero crear, ver, editar y eliminar mis tareas académicas, para poder llevar control de mis pendientes por materia.

- Escenario: crear una tarea → POST /api/tareas con título, materia y fecha válidos → la tarea queda visible en el listado paginado del propio usuario.
- Escenario: listar tareas paginadas → GET /api/tareas?page=0&size=10 → exactamente 10 tareas + metadatos de paginación.
- Escenario: editar una tarea propia → PUT /api/tareas/{id} → 200 y cambios reflejados.

Trazabilidad: REQ-F-003 → CU-02. Verificado por TareaControllerTest (8 pruebas, incluido el aislamiento entre usuarios corregido de OWASP A01).

HU-03 — Dashboard con resumen académico

Como estudiante, quiero ver de un vistazo mis tareas pendientes, mi promedio general y mis avisos sin leer al entrar al sistema, para no tener que revisar cada sección por separado.

- Escenario: ver resumen académico → GET /api/dashboard → tareasPendientes, avisosNoLeidos y promedioGeneral reflejan el estado real de la base de datos.
- Escenario: marcar avisos como leídos en bloque → PUT /api/dashboard/avisos/marcar-leidos → avisosMarcados igual a la cantidad actualizada, y una consulta posterior confirma avisosNoLeidos en 0.

Trazabilidad: REQ-F-004, REQ-F-005 → CU-03. Esta historia es la que justifica el uso de procedimientos almacenados (ver sección 4.2), por requerir agregación sobre tres tablas en una sola proyección. Verificado por DashboardControllerTest (4 pruebas).

HU-04 — Consulta al asistente conversacional (pendiente)

Como estudiante, quiero preguntarle al asistente virtual cosas como '¿cuántas tareas tengo pendientes?' en lenguaje natural, para no tener que navegar por el dashboard.

Estado: pendiente de implementación. El backend (ChatbotController/ChatbotService) no existe todavía en el repositorio; se documenta el criterio de aceptación previsto (respuesta predefinida por palabra clave, o consulta a la Gemini API si no hay coincidencia) para guiar la implementación en la Entrega Final. Trazabilidad: REQ-F-006 (pendiente) → CU-04 (pendiente).

4.2 Catálogo de procedimientos almacenados

Las tres funciones PL/pgSQL del sistema están documentadas en docs/basedatos/CATALOGO-SP.md conforme al Bloque A.2.2. Ninguna construye SQL dinámico ni concatena entrada de usuario; todas reciben parámetros nombrados y tipados.

Función	Tipo	Motivo (no ORM)	Endpoint
fn_obtener_resumen_academico	Retorna 1 fila	Agregación (COUNT/AVG/MIN) sobre 3 tablas en una proyección sin entidad JPA	GET /api/dashboard
fn_listar_tareas_pendientes	Retorna conjunto de filas	JOIN entre tareas y materia, más columna calculada (días restantes)	GET /api/dashboard
fn_marcar_avisos_leidos	Retorna escalar, con escritura	Actualización masiva por criterio distinto a la clave primaria	PUT /api/dashboard/avisos/mar

Función	Tipo	Motivo (no ORM)	Endpoint
			car-leídos

4.2.1 Parámetros y tablas afectadas (detalle)

Función	Parámetro entrada	Columnas / valor de salida	Tablas afectadas
fn_obtener_resumen_academico	p_id_usuario INTEGER	tareas_pendientes, promedio_general NUMERIC(4,2), avisos_no_leídos, proxima_entrega DATE	tareas, calificaciones, avisos (solo lectura)
fn_listar_tareas_pendientes	p_id_usuario INTEGER	id_tarea, titulo, materia, fecha_entrega, dias_restantes	tareas, materia (solo lectura, JOIN)
fn_marcar_avisos_leídos	p_id_usuario INTEGER	INTEGER (filas actualizadas, vía GET DIAGNOSTICS)	avisos (escritura, UPDATE)

Convenciones generales del catálogo: nomenclatura fn_<verbo>_<sustantivo>.sql para funciones y sp_<verbo>_<sustantivo>.sql para procedimientos; todas versionadas en db/procs/ y aplicadas junto con el esquema base al levantar el contenedor de PostgreSQL (Bloque B).

4.3 Casos de uso (plantilla de Cockburn)

Los flujos críticos del sistema se documentan también como casos de uso completos siguiendo la plantilla de Cockburn (niveles de precisión 1 a 4: actor/objetivo, escenario principal de éxito, condiciones de extensión, pasos de manejo de extensión).

CU-01 — Autenticación segura

Actor principal / objetivo: usuario registrado (estudiante o docente) que inicia sesión de forma segura.

Precondición: el usuario ya está registrado. Garantía de éxito: recibe un JWT válido y accede a endpoints protegidos hasta expirar o cerrar sesión.

- Escenario principal: login → búsqueda por email → verificación BCrypt → generación de JWT (claims sub, rol, nombre, iat, exp) → el frontend adjunta el token como Bearer en cada petición protegida → JwtAuthFilter valida firma, expiración y ausencia en la blacklist de Redis.
- Extensión 3a/3b (email no registrado / contraseña incorrecta): 400. El propio documento de caso de uso señala como mejora identificada, no implementada, distinguir menos entre ambos casos en el mensaje al usuario final para reducir el riesgo de enumeración de usuarios (aunque el mensaje de error ya es genérico a nivel de API, según se documentó en la sección 6.2).
- Extensión 7a/8a/8b (token ausente, expirado, o en blacklist): 403, el frontend limpia sesión y redirige al login.
- Extensión 8c (Redis no disponible): no bloquea la autenticación — política fail-open documentada en ADR-002.

CU-02 — Gestión de tareas académicas

Actor principal / objetivo: estudiante autenticado que crea, consulta, edita o elimina sus propias tareas.

- Escenario principal (crear): formulario → POST /api/tareas con JWT → JwtAuthFilter puebla el SecurityContext → TareaRepository persiste la fila asociada al id_usuario del token → respuesta con la tarea creada.
- Escenario alternativo (listar paginado): GET /api/tareas?page=0&size=10, filtrado por id_usuario — CRUD elemental según A.2.1, no requiere procedimiento almacenado.

Hallazgo de documentación desactualizada: el archivo original de este caso de uso (CU-02-gestion-tareas.md) señala como condición de extensión general 'pendiente de verificación explícita' que el sistema rechace intentos de editar/eliminar tareas de otro usuario. Esa verificación ya está implementada y probada durante esta entrega (corrección del hallazgo OWASP A01, ver secciones 2 y 6.2, con las pruebas usuarioBNoPuedeEditarTareaDeUsuarioADevuelve403 y usuarioBNoPuedeEliminarTareaDeUsuarioADevuelve403 de TareaControllerTest). Se recomienda actualizar el archivo fuente del caso de uso para reflejar este cierre antes de la Entrega Final.

CU-03 — Consulta del dashboard académico

Actor principal / objetivo: estudiante autenticado que ve, en una sola pantalla, tareas pendientes, promedio y avisos.

- Escenario principal: GET /api/dashboard → DashboardController resuelve el usuario desde el email del token (nunca desde un parámetro del cliente) → DashboardRepository invoca fn_obtener_resumen_academico y fn_listar_tareas_pendientes → el frontend renderiza las tarjetas.
- Escenario alterno (marcar avisos leídos): PUT /api/dashboard/avisos/marcar-leidos → invoca fn_marcar_avisos_leidos → devuelve la cantidad actualizada.
- Extensión 4b (usuario sin datos aún, cuenta nueva): las funciones SQL devuelven 0/NULL de forma segura; el frontend muestra un estado vacío coherente ('Aún sin calificaciones'), verificado en el código actual de cargarDashboard().

CU-04 — Consulta al asistente conversacional (pendiente)

Este caso de uso, declarado desde la Entrega 1A como diferenciador central del proyecto, describe el comportamiento deseado del chatbot híbrido: primero buscar coincidencia por palabra clave en la tabla respuestas_bot (activo=true) y, sin coincidencia, construir un prompt y consultar la Gemini API. El documento fuente deja explícitamente abiertas tres decisiones de diseño aún no tomadas: cómo priorizar múltiples palabras clave coincidentes, qué hacer si la Gemini API no responde o excede un tiempo límite (se recomienda un timeout explícito de 8-10 segundos), y si filtrar contenido de la respuesta generada antes de mostrarla. Ninguna línea de código de ChatbotController/ChatbotService existe todavía en el repositorio.

4.4 Nota sobre organización de archivos

Durante la revisión de esta entrega se detectó que la carpeta docs/requisitos/casos-de-uso/ tiene una estructura anidada inconsistente (por ejemplo, docs/requisitos/casos-de-uso/gestion de tareas/requisitos/CU-03-dashboard.md, con espacios en los nombres de carpeta y una subcarpeta requisitos duplicada dentro de casos-de-uso), probablemente resultado de mover archivos entre carpetas durante el desarrollo. El contenido de los cuatro casos de uso existe y es correcto, pero se recomienda aplanar esa estructura antes de la Entrega Final para facilitar la revisión.

4.5 Matriz de trazabilidad completa

De los 8 requisitos funcionales/no funcionales con prioridad Must, 3 están en estado verificado, 3 en estado implementado y 2 en estado pendiente (REQ-F-006, el chatbot, y REQ-NF-001, el benchmark específico del dashboard). Los 2 requisitos Should están implementado/verificado. La matriz cubre el 100% de los requisitos Must vigentes.

ID	Tipo	Prioridad	Módulo	Tipo acceso	Estado
REQ-F-001	Funcional	Must	AuthController; AuthService	CRUD-ORM	verificado
REQ-F-002	Funcional	Must	JwtAuthFilter	CRUD-ORM	implementado
REQ-F-003	Funcional	Must	TareaController; TareaRepository	CRUD-ORM	implementado
REQ-F-004	Funcional	Must	DashboardController; DashboardRepository	SP	implementado
REQ-F-005	Funcional	Should	DashboardController; DashboardRepository	SP	implementado

ID	Tipo	Prioridad	Módulo	Tipo acceso	Estado
					ntado
REQ-F-006	Funcional	Must	(sin implementar — chatbot)	—	pendiente
REQ-NF-001	No funcional	Must	DashboardController	SP	pendiente
REQ-NF-002	No funcional	Must	fn_obtener_resumen_academico; fn_listar_tareas_pendientes; fn_marcar_avisos_leidos	SP	verificado
REQ-NF-003	No funcional	Should	RedisService	CRUD-ORM	verificado
REQ-NF-004	No funcional	Must	docker-compose.yml; Makefile; db/schema.sql	—	verificado

5. Protocolo experimental

5.1 Rendimiento (k6)

Herramienta: k6 v2.1.0. Configuración: 50 VUs, con ramp-up de 10s, carga sostenida de 30s y ramp-down de 5s (k6/opts.js). Tres corridas independientes contra GET /api/tareas?page=0&size=10, autenticando una vez por corrida (setup()) con el usuario semilla (admin@uteq.edu.ec) y reutilizando el token JWT vía cabecera Authorization: Bearer en cada iteración. Certificado autofirmado (backend en https://localhost:8443), por lo que se usó insecureSkipTLSVerify.

5.2 Seguridad (OWASP)

Auditoría manual con curl contra los 6 controles mínimos exigidos (A01, A02, A03, A05, A07, A09), con la salida completa de cada comando archivada en docs/mediciones/sec/. El control A01 se verificó contra el propio hallazgo corregido durante esta entrega (aislamiento de tareas entre usuarios).

5.3 Cobertura (JaCoCo)

mvn clean test ejecutado localmente contra una instancia real de PostgreSQL (no una base en memoria), con el agente JaCoCo 0.8.12 adjunto vía Maven. 23 pruebas JUnit 5 + MockMvc, 0 fallos, 0 errores. El reporte HTML/XML/CSV se genera directamente en docs/mediciones/jacoco/. Nota metodológica: dado que la imagen Docker del backend se construye con mvn package -DskipTests (ver Dockerfile, Bloque B.1, para separar el build de producción de la ejecución de pruebas), este reporte se regenera ejecutando las pruebas de forma local, no dentro del contenedor.

5.4 Accesibilidad y calidad web (Lighthouse)

Lighthouse CLI 13.4.1 (vía @lhci/cli), throttling method "simulate" con perfil de red aproximado a Slow 4G (RTT 150ms, throughput 1638.4kbps, CPU slowdown 4x), ejecutado contra http://localhost/index.html servido por el contenedor Nginx. Nota metodológica: se usó lighthouse directo en vez de lhci autorun por un defecto conocido de chrome-launcher en Windows (EPERM al limpiar la carpeta temporal del navegador tras cerrar Chrome) que interrumpe el proceso de lhci después de generar el reporte; el JSON de resultados se confirmó guardado correctamente antes del error de limpieza, por lo que la medición en sí no se vio afectada.

5.5 Usabilidad (SUS) — pendiente

No se ejecutó en este ciclo. El protocolo exige un mínimo de 10 participantes externos al equipo con consentimiento informado firmado (plantilla ya preparada en docs/etica/consentimientos/plantilla.md). Dada la restricción de tiempo entre la publicación de esta guía y la fecha de cierre, no fue posible reclutar y coordinar sesiones con participantes reales. Queda como tarea explícita para la Entrega Final.

El instrumento previsto es el System Usability Scale (SUS) de Brooke, compuesto por 10 afirmaciones que el participante puntúa en una escala de 5 puntos (1 = totalmente en desacuerdo, 5 = totalmente de acuerdo), alternando

afirmaciones positivas y negativas para reducir sesgo de aquiescencia. La tarea común de onboarding prevista es: login, alta de un registro, edición, eliminación lógica, logout — exactamente el flujo que ya se verificó funcionando de punta a punta en la sección 6.6 de este informe, lo cual reduce el riesgo de que la sesión de usabilidad se vea interrumpida por un defecto técnico no relacionado con la experiencia de uso.

La puntuación SUS se calcula normalizando cada respuesta (ítems impares: puntuación – 1; ítems pares: 5 – puntuación), sumando los 10 valores normalizados y multiplicando por 2.5, para obtener un puntaje de 0 a 100 por participante. El reporte agregado previsto incluye media, desviación típica e intervalo de confianza al 95% sobre el conjunto de participantes, siguiendo el mismo estándar estadístico ya aplicado en el Bloque C.1 (rendimiento).

6. Resultados por bloque

6.1 Rendimiento

Corrida	avg (ms)	p50 (ms)	p90 (ms)	p95 (ms)	p99 (ms)	Error rate
1	3.22	3.21	3.60	3.71	4.05	0.00%
2	3.29	3.27	3.63	3.74	4.29	0.00%
3	3.49	3.41	4.01	4.21	4.93	0.00%

Umbral objetivo: p95 < 500ms (caché frío). Resultado: 3.71–4.21ms en las tres corridas, muy por debajo del umbral. Tasa de errores HTTP ≥500: 0% en las tres corridas. Como se documentó en la sección 5.1, estas mediciones representan un único escenario (sin caché activa), no una comparación caliente/frío.

6.2 Seguridad OWASP

Los 6 controles mínimos están evidenciados con salida real de curl en docs/mediciones/sec/, más las pruebas automatizadas de AuthControllerTest y TareaControllerTest que verifican el mismo comportamiento de forma reproducible en cada ejecución de la suite.

- A01 (control de acceso): un usuario autenticado que solicita una tarea de otro usuario recibe 403 Forbidden. Verificado con evidencia curl y con las pruebas usuarioBNoPuedeVerTareaDeUsuarioADevuelve403 y equivalentes para editar/eliminar.
- A02 (fallo criptográfico): backend accesible en https://localhost:8443 con certificado autofirmado, TLS activo y verificado end-to-end durante esta entrega (ver corrección del keystore y del mapeo de puerto en docker-compose.yml).
- A03 (inyección): un payload ' OR '1'='1 en el campo email de login recibe 422 con ProblemDetails, rechazado por el patrón de validación antes de llegar a cualquier consulta. Verificado con evidencia curl y con la prueba automatizada loginConPayloadDeInyeccionEnEmailDevuelve422.
- A05 (configuración de seguridad): la evidencia capturada (docs/mediciones/sec/A05-headers-seguridad.txt) confirma la presencia de X-Content-Type-Options: nosniff, X-Frame-Options: DENY y Content-Security-Policy: default-src 'self'; frame-ancestors 'none'. Hallazgo pendiente: esa captura corresponde a una respuesta por HTTP plano (puerto 8080, previo a la activación de TLS en 8443), por lo que no incluye el encabezado Strict-Transport-Security exigido por el control A05 completo; se recomienda recapturar esta evidencia contra https://localhost:8443 antes de la Entrega Final.
- A07 (identificación y autenticación): al sexto intento fallido consecutivo desde la misma IP, el sistema responde 429 Too Many Requests. Verificado con evidencia curl y con la prueba automatizada seisIntentosFallidosConsecutivosDevuelve429EnElSexto, agregada durante esta entrega.
- A09 (registro y monitoreo): cada intento de login, exitoso o fallido, se registra con IP, timestamp y el sub (email) involucrado, por ejemplo: "LOGIN FALLIDO | ip=... | sub=tarea.usuarioA@uteq.edu.ec | motivo=contrasena incorrecta | intento=1", evidencia archivada en docs/mediciones/sec/A09-registro-logs.txt.

6.3 Cobertura de pruebas

Métrica	Cobertura
Instrucciones	81.6%
Líneas	80.7%
Ramas (branches)	66.1%
Pruebas ejecutadas	23 (0 fallos, 0 errores)

Supera ampliamente el mínimo de 60% exigido para esta entrega, y ya alcanza el objetivo de 70% fijado para la Entrega Final.

6.4 Accesibilidad y calidad web (Lighthouse)

Categoría	Umbral mínimo	Resultado
Performance	≥ 80	91
Accessibility	≥ 90	100
Best Practices	≥ 90	96
SEO	≥ 90	90

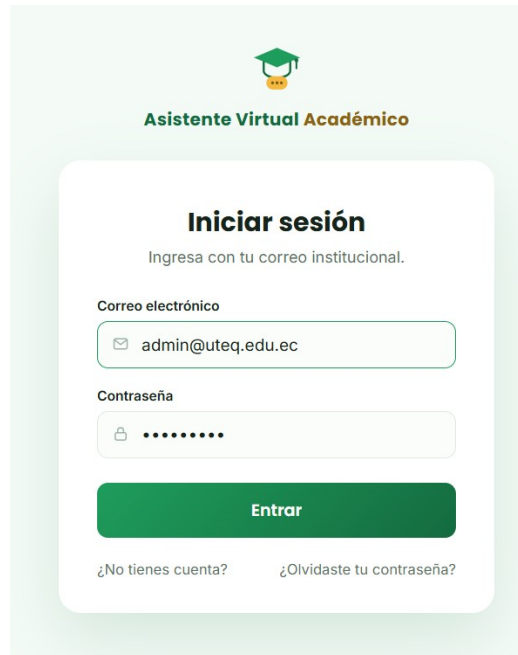
La primera corrida detectó dos hallazgos reales de accesibilidad en index.html (contraste de color insuficiente en el texto secundario y en el acento dorado del logo, y ausencia de un landmark <main>), ambos corregidos durante esta entrega; la re-auditoría confirmó Accessibility 100/100.

6.5 Usabilidad (SUS)

Pendiente — ver sección 5.5. No se reporta puntuación SUS en esta entrega.

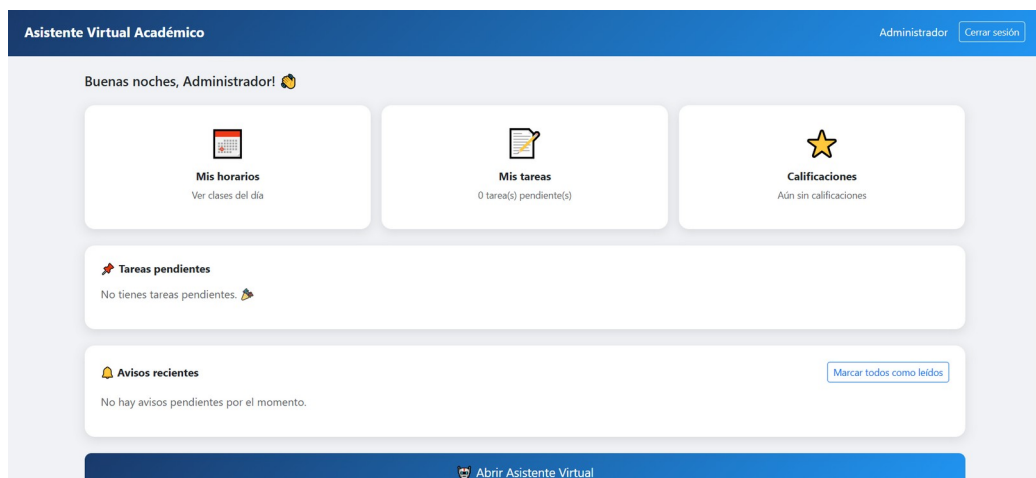
6.6 Evidencia visual del sistema en funcionamiento

Durante la revisión de esta entrega se descubrió y corrigió un defecto crítico no detectado hasta entonces: el frontend llamaba a la API con rutas relativas (por ejemplo `fetch('/api/auth/login')`), que al servirse desde nginx en el puerto 80 nunca llegaban al backend (puerto 8443) — el login fallaba siempre con 'Error de conexión con el servidor' pese a que todas las pruebas automatizadas, curl y k6 contra el backend directo pasaban sin problema. La causa raíz combinó dos defectos: rutas relativas en el frontend en vez de absolutas, y ausencia total de configuración CORS en SecurityConfig, que bloqueaba el preflight OPTIONS de cualquier endpoint autenticado antes de que Spring MVC pudiera aplicar las anotaciones `@CrossOrigin` ya presentes en los controladores. Ambos se corrigieron durante esta entrega; las capturas siguientes documentan el flujo funcionando de punta a punta después del fix.



The login form is titled "Asistente Virtual Académico" and "Iniciar sesión". It prompts the user to "Ingresa con tu correo institucional." and includes fields for "Correo electrónico" (admin@uteq.edu.ec) and "Contraseña" (masked with dots). A green "Entrar" button is at the bottom, along with links for "¿No tienes cuenta?" and "¿Olvidaste tu contraseña?".

Figura 4 — Login funcionando (index.html)



The dashboard is titled "Asistente Virtual Académico" and shows the user as "Administrador". It displays a greeting "Buenas noches, Administrador!" and three main sections: "Mis horarios" (Ver clases del día), "Mis tareas" (0 tarea(s) pendiente(s)), and "Calificaciones" (Aún sin calificaciones). Below these are "Tareas pendientes" (No tienes tareas pendientes.) and "Avisos recientes" (No hay avisos pendientes por el momento.) with a "Marcar todos como leídos" button. A footer button says "Abrir Asistente Virtual".

Figura 5 — Dashboard cargando datos reales tras el fix de CORS

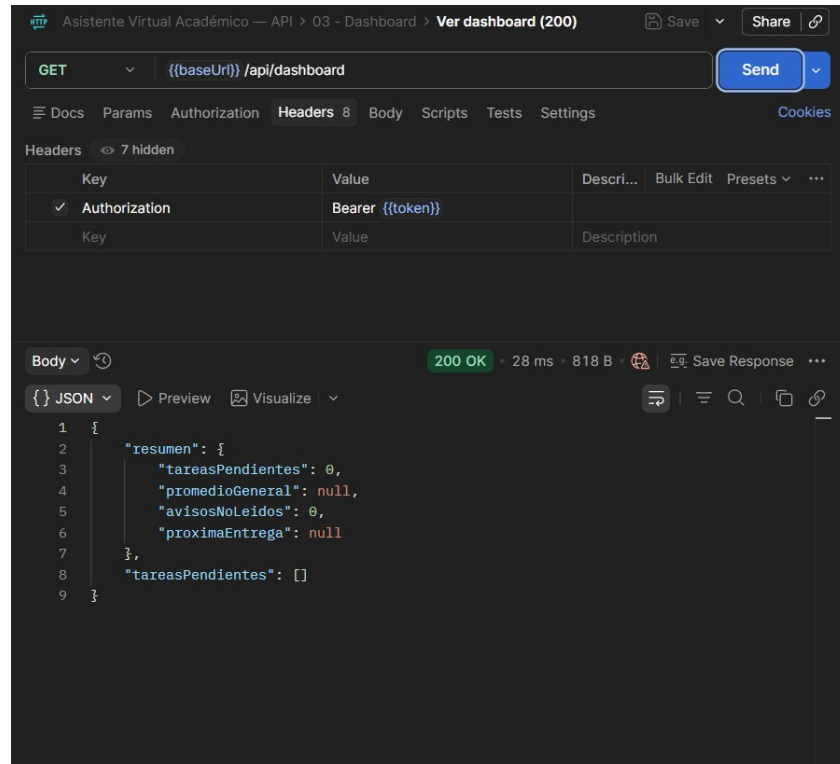


Figura 6 — GET /api/dashboard vía Postman, 200 OK con JSON real

6.7 Colección Postman

Se generó una colección Postman de referencia (docs/postman/coleccion.json) con 22 peticiones organizadas en 4 carpetas, cubriendo casos de éxito, validación (422), credenciales incorrectas (400), no autorizado (403), no encontrado (404) y límite de intentos (429) contra los tres controladores reales del sistema.

Carpeta	Peticiones	Casos cubiertos
01 - Auth	7	Login (200), inyección en email (422), contraseña incorrecta (400), email inexistente (400), registro (200), logout sin/con token (400/200)
02 - Tareas	9	Listar sin token (403), listar paginado (200), crear (200), obtener por id (200), obtener inexistente (404), editar propia (200), editar/eliminar de otro usuario (403), eliminar propia (200)
03 - Dashboard	4	Ver sin token (403), ver dashboard (200), marcar avisos leídos (200), marcar sin token (403)
04 - Seguridad	2	Rate limit al 6to intento (429), cabeceras de seguridad

La captura de la Figura 6 (sección 6.6) corresponde a la petición 'Ver dashboard (200)' de esta misma colección, ejecutada contra el sistema ya corregido.

7. Amenazas a la validez

- El benchmark de rendimiento se ejecutó con el generador de carga (k6) y el sistema bajo prueba en la misma máquina física, sin latencia de red real; los tiempos de respuesta reportados no son representativos de un despliegue en producción con cliente y servidor separados.
- No existe comparación caché caliente/frío porque el sistema, en su estado actual, no implementa una capa de caché de respuestas HTTP (ver limitación documentada en la sección 2.2).
- La auditoría de Lighthouse se ejecutó en una sola corrida (no las tres corridas independientes recomendadas para reducir varianza de medición), por una limitación de tiempo, no metodológica.

- No hay evidencia de usabilidad con usuarios reales (SUS) en este ciclo; toda afirmación sobre la facilidad de uso del sistema es, por ahora, cualitativa y no está respaldada empíricamente.
- La cobertura de pruebas se midió ejecutando la suite localmente contra una instancia de PostgreSQL en el host, no contra el contenedor Docker desplegado; se asume paridad de comportamiento entre ambos entornos, pero no se verificó de forma independiente.
- El hallazgo de la sección 6.6 (rutas relativas rotas + CORS ausente) permaneció sin detectar durante buena parte del desarrollo porque toda la evidencia automatizada (curl, k6, Postman, JUnit) se ejecuta contra el backend directamente, sin pasar por el frontend ni por un navegador real; esto es una amenaza a la validez del propio proceso de verificación, no solo del sistema, y sugiere que la Entrega Final debería incorporar al menos una prueba automatizada de extremo a extremo (por ejemplo con Selenium o Playwright) que ejercite el flujo completo navegador → frontend → backend.

8. Declaración de contribuciones (CRediT)

Fajardo Montes Michael Xavier — Conceptualization, Software (backend, frontend, Docker/reproducibilidad), Investigation (diagnóstico y corrección de defectos), Writing – original draft (SRS, historias de usuario, bitácora de observaciones, este informe), Project administration.

Rivera Suárez Jonathan Javier — Documentation (los 6 ADR), Software (diagramas C4 en Structurizr DSL), Writing – original draft (tabla ISO/IEC 25010); asume tareas adicionales tras la baja de Castro Palma.

Castro Palma Justyn Moisés — Se dio de baja de la materia durante el desarrollo del proyecto. Contribuyó a las Entregas 1A/1B y a los diagramas iniciales del equipo antes de su retiro.

9. Declaración ética

1. Fuentes de datos y su licencia. Todos los datos utilizados durante el desarrollo (usuarios, materias, tareas, calificaciones, avisos) son datos sintéticos de prueba creados por el propio equipo; no se ha usado, en ningún momento, una base de datos real de estudiantes de la UTEQ ni de ninguna otra institución. El código fuente se licencia bajo MIT.

2. Tratamiento de datos personales. El sistema, en su estado v0.9.0-rc, no gestiona datos de estudiantes reales; los campos de información personal identificable están poblados únicamente con datos ficticios en todo el desarrollo y las pruebas. De llegar a desplegarse con datos reales, el sistema ya implementa como mínimo: contraseñas con hash BCrypt (nunca en texto plano), autenticación stateless vía JWT sin exponer credenciales entre peticiones, y separación de acceso a datos por rol (en desarrollo, ver ADR-006).

3. Consentimiento informado (pruebas de usabilidad). Para las pruebas SUS del Bloque C.3 (participantes humanos reales), el equipo cuenta con una plantilla de consentimiento informado (docs/etica/consentimientos/plantilla.md) que cada participante debe firmar antes de participar. El formulario explica el propósito de la prueba, que la participación es voluntaria y puede abandonarse en cualquier momento sin consecuencias, y que las respuestas se anonimizan e identifican solo por código (P01, P02, ...), nunca por nombre.

4. Ausencia de datos identificables en el repositorio público. El repositorio público no contiene formularios de consentimiento firmados (quedan archivados solo localmente), nombres reales de participantes de pruebas de usabilidad, ni ninguna credencial real de producción (las de db/seed.sql y la documentación son explícitamente de desarrollo, documentadas como tales).

5. Alcance declarado. Este proyecto es un Proyecto Fin de Curso académico de la UTEQ, sin fines comerciales ni despliegue en producción con usuarios reales durante el período de esta entrega.

Declaración completa en docs/etica/ETHICS.md.

10. Próximos pasos hacia la Entrega Final

La Tercera Entrega deja al equipo en el estado v0.9.0-rc, candidato a versión estable. La transición a v1.0.0 debe absorber la evaluación de esta entrega y, en particular, atender los siguientes puntos ya identificados durante este ciclo:

- Migrar el JWT de Bearer token a cookie HttpOnly+Secure+SameSite=Strict, o documentar formalmente en el ADR-002 la decisión de mantener Bearer token con su justificación.
- Implementar caché de respuestas HTTP con Redis (TTL externo, hit ratio medido) en el endpoint de listado, tal como reconoce el propio ADR-004 como deuda técnica pendiente.
- Añadir los claims aud y jti al JWT, conforme a RFC 7519, según lo anota el ADR-002.
- Introducir un DTO de respuesta para Usuario que excluya el campo de hash de contraseña.
- Ejecutar la prueba de usabilidad SUS con al menos 10 participantes reales, usando la plantilla de consentimiento ya preparada.
- Recapturar la evidencia del control OWASP A05 contra el endpoint HTTPS (puerto 8443) para incluir el encabezado Strict-Transport-Security.
- Exportar los diagramas C4 (L1–L3) de Structurizr DSL a PNG e incorporarlos a este informe.
- Elevar la cobertura de pruebas del 80.7% actual hacia el 70% ya superado, ampliando la cobertura de ramas (66.1% actual) en los controladores.
- Completar el chatbot (HU-04 / CU-04 / REQ-F-006), única funcionalidad Must sin implementar en el estado actual.

Anexo A — Historial de commits

El repositorio cuenta con 61 commits al momento de este borrador, muy por encima del mínimo de 30 commits granulares exigido por el criterio C10 de la rúbrica. A continuación se listan los hitos principales, en orden cronológico, como evidencia de la evolución incremental del proyecto (no es la lista completa de commits, que puede consultarse íntegra con git log en el repositorio).

Hito	Commit (resumen)
Estructura inicial	feat: initial project structure - Spring Boot setup
Autenticación JWT	feat: modulo de autenticacion JWT registro y login
Frontend inicial	feat: agregar frontend HTML login registro dashboard y chatbot
Docker Compose	feat: agregar Docker Compose y Dockerfile
CRUD de Tareas	agregar CRUD de Tarea con paginacion
Flyway	feat: agregar Flyway para migraciones de base de datos
Blacklist Redis	feat: agregar Redis blacklist para logout de tokens JWT
Cierre de observaciones 1A/1B	docs(arquitectura): agrega justificacion de tecnologias del C4 N1 (OBS-01)
Reproducibilidad Docker	feat(infra): reproducibilidad automatica con Docker (Bloque B)
SRS y requisitos	docs(requisitos): borrador inicial del SRS (ISO/IEC/IEEE 29148)
Ética y licenciamiento	docs(etica): declaracion etica del proyecto; docs: LICENSE MIT; CITATION.cff
Matriz de trazabilidad	docs(trazabilidad): matriz end-to-end requisitos-codigo-pruebas
Hallazgo y fix OWASP A01	fix(security): TareaController no filtraba por usuario (OWASP A01)
Controles OWASP restantes	feat(security): controles OWASP A01, A05, A07, A09; HTTPS/TLS (A02)
Fix Makefile	fix(infra): corrige Makefile (tenia contenido de Dockerfile por error)
Scripts de auditoría CI	feat(ci): agrega scripts de auditoria (trazabilidad, SQL dinamico)
Benchmark k6	feat(perf): agrega benchmark k6 (C.1) con 3 corridas + reporte agregado
Fix accesibilidad + Lighthouse	fix(a11y): corrige contraste de color y landmark main; auditoria Lighthouse
Cobertura 80.7%	test(auth): agrega pruebas de AuthController; cobertura sube a 80.7%
Fix diagramas C4	fix(arquitectura): corrige sintaxis DSL de C4 Nivel 3 y exporta PNG
DOI de Zenodo	docs: agrega DOI de Zenodo en README y CITATION.cff

Anexo B — Glosario

Término	Significado
JWT	JSON Web Token: token firmado que representa afirmaciones seguras entre dos partes (RFC 7519)
ADR	Architecture Decision Record: documento corto que registra una decisión arquitectónica, su contexto y sus consecuencias
C4	Modelo de diagramación de arquitectura en 4 niveles: Contexto, Contenedores, Componentes, Código
SP	Stored Procedure / función almacenada: bloque de código SQL compilado en el motor de base de datos
TTL	Time To Live: tiempo de vida de una entrada en caché antes de expirar automáticamente
ORM	Object-Relational Mapping: técnica que traduce objetos del código a filas de una base de datos relacional
SUS	System Usability Scale: instrumento estandarizado de 10 preguntas para medir usabilidad percibida
p95 / p99	Percentil 95 / 99: el 95% (o 99%) de las mediciones fueron iguales o menores a este valor
DTO	Data Transfer Object: objeto usado para transportar datos entre capas, distinto de la entidad de persistencia
CRediT	Contributor Roles Taxonomy: taxonomía estandarizada de 14 roles para declarar contribuciones de autoría

Anexo C — Estructura del repositorio y limpieza pendiente

Estructura de primer y segundo nivel del repositorio al momento de este borrador:

Ruta	Contenido
README.md, LICENSE, CITATION.cff, CHANGELOG.md, VERSIONING.md	Metadatos de publicabilidad (Bloque E)
Makefile, docker-compose.yml, Dockerfile, .env.example	Reproducibilidad automática (Bloque B)
db/schema.sql, db/seed.sql, db/procs/	Esquema y funciones SQL versionadas
docs/adr/, docs/arquitectura/	6 ADR + diagramas C4 (DSL y PNG)
docs/requisitos/, docs/trazabilidad/	SRS, historias, casos de uso, matriz
docs/mediciones/	Evidencia empírica: perf, sec, jacoco, lighthouse
docs/etica/, docs/basedatos/, docs/postman/	Ética, catálogo SP, colección Postman
k6/, scripts/, lighthousec.js	Herramientas de benchmark y auditoría
src/main/, src/test/	Código fuente Java y pruebas JUnit

Pendiente de limpieza (no bloqueante, pero recomendado antes de la Entrega Final): el archivo de contribuciones sigue nombrado 'CONTRIBUTORS .md' (con un espacio antes de la extensión), lo cual puede hacer que verificaciones automatizadas que busquen exactamente 'CONTRIBUTORS.md' no lo encuentren. Además, existe una carpeta database/ en la raíz (con un schema.sql y una imagen suelta) que duplica el propósito de db/, y una carpeta evidence/diagramas/ fuera de la estructura documentada por la guía — ambas parecen residuos de reorganizaciones anteriores del repositorio y no forman parte de la estructura mínima exigida.

Anexo D — Mapeo de puertos y orígenes CORS

Tabla de referencia del estado final de red del sistema, documentada aquí porque dos de sus valores fueron corregidos durante esta entrega (ver secciones 2.2 y 6.6) y son fáciles de romper de nuevo por error en cambios futuros.

Servicio	Puerto host	Puerto interno	Notas
Frontend (nginx)	80	80	Sirve HTML/CSS/JS estático; no proxya /api/*
Backend (Spring Boot)	8443	8443	HTTPS con certificado autofirmado (keystore.p12); antes mapeado erróneamente a 8080
PostgreSQL	5432	5432	Expuesto para depuración local; no

Servicio	Puerto host	Puerto interno	Notas
			requerido por el frontend
Redis	6379	6379	Expuesto para depuración local; no requerido por el frontend

Orígenes permitidos por CORS (@CrossOrigin) en cada controlador, tras la corrección: AuthController y TareaController permiten http://localhost:8080 y http://localhost; DashboardController, que antes no tenía ninguna anotación de CORS, ahora permite los mismos dos orígenes. La configuración de Spring Security (SecurityConfig) habilita .cors() y permite explícitamente las peticiones OPTIONS (preflight) sin autenticación — sin ese cambio, ninguna anotación @CrossOrigin tiene efecto sobre endpoints protegidos, como se documentó en la sección 6.6.

Anexo E — Diccionario de datos (esqueleto pendiente)

El Bloque E.3 de la guía exige un diccionario de datos (docs/mediciones/DATA-DICTIONARY.md) que registre, para cada archivo crudo de mediciones, el nombre de la variable, su tipo de dato, unidad, rango esperado y significado. Ese archivo no existe todavía en el repositorio; se deja aquí el esqueleto de las variables ya generadas por los archivos de evidencia existentes, como punto de partida.

Archivo fuente	Variable	Tipo / unidad	Significado
k6-run{1,2,3}.json	http_req_duration (avg/p90/p95/p99)	float, ms	Tiempo de respuesta del endpoint bajo prueba
k6-run{1,2,3}.json	http_req_failed	float, [0,1]	Tasa de peticiones con error HTTP
jacoco.csv	LINE_COVERED / LINE_MISSED	entero, líneas	Líneas ejercitadas / no ejercitadas por las pruebas
lhci-*.json	categories.*.score	float, [0,1]	Puntaje normalizado por categoría de Lighthouse
sus-raw.csv (pendiente)	item_01..item_10	entero, [1,5]	Respuesta de cada participante a cada afirmación SUS

Anexo F — Matriz de cumplimiento por bloque de la guía

Síntesis del estado de cada bloque exigido por la guía de la Tercera Entrega, con la ubicación de su evidencia dentro de este informe o del repositorio.

Bloque	Estado	Evidencia
0 — Observaciones 1A/1B	Cerrado (100%)	Sección 1; docs/observaciones/OBSERVACIONES.md; tag v0.7.1
A.1 — Consolidación funcional	Parcial	Sección 2; JWT vía Bearer (no cookie), sin caché de respuestas
A.2 — Acceso a datos (ORM/SP)	Cerrado	Sección 4.2; docs/basedatos/CATALOGO-SP.md; sin SQL dinámico (Bloque C.2 script)
A.3 — Ingeniería de requisitos	Cerrado	Secciones 4.0–4.5; SRS, historias, casos de uso, matriz
B — Reproducibilidad	Cerrado y verificado	Sección 6.6; make up de punta a punta incluido navegador
C.1 — Rendimiento	Cerrado	Sección 6.1; docs/mediciones/perf/
C.2 — Seguridad OWASP	Cerrado (6/6 controles)	Sección 6.2; docs/mediciones/sec/ (A05 con hallazgo pendiente)
C.3 — Usabilidad (SUS)	Pendiente, justificado	Sección 5.5; sin participantes reales en este ciclo
C.4 — Cobertura	Cerrado, supera objetivo	Sección 6.3; 80.7% líneas, 23 pruebas
C.5 — Lighthouse	Cerrado	Sección 6.4; los 4 umbrales superados
D — Documentación arquitectónica	Cerrado	Sección 3; 6 ADR, C4 L1-L3, ISO 25010
E — Publicabilidad	Cerrado salvo diccionario de datos	README, CITATION.cff, DOI Zenodo; Anexo E pendiente
F — Ética	Cerrado	Sección 9; docs/etica/ETHICS.md

Referencias

International Organization for Standardization, International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers. ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering.

ISO/IEC 25010:2011 — Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE).

Jones, M., Bradley, J., Sakimura, N. JSON Web Token (JWT). RFC 7519, IETF, 2015.

Nottingham, M., Wilde, E. Problem details for HTTP APIs. RFC 7807, IETF, 2016.

OWASP Foundation. OWASP Top 10:2021 — The ten most critical web application security risks, 2021.

Nygard, M. Documenting architecture decisions. Cognitect Blog, 2011.

Brooke, J. SUS: A 'quick and dirty' usability scale. En Usability Evaluation in Industry, Taylor and Francis, 1996.

National Information Standards Organization (NISO). ANSI/NISO Z39.104-2022, CRediT, Contributor Roles Taxonomy, 2022.

Preston-Werner, T. Semantic Versioning 2.0.0, 2013.

Wilkinson, M. D. et al. The FAIR guiding principles for scientific data management and stewardship. Scientific Data, 3:160018, 2016.

Association for Computing Machinery. Artifact review and badging — version 1.1, 2020.